

Necessity for a New extensibility model

- Extensibility enables customers to create a competitive advantage by customizing their business processes and allows partners to enrich ERP with tailor-made solutions.
- classical extensibility is not consistent with cloud native
- SAP S/4 HANA cloud extensibility model helps customers keep and maintain a clean core even when building extensions.

SAP S/4 HANA cloud extensibility model

Three principles -

- Extensions can only be implemented internally within SAP S/4 HANA cloud as on-stack extensions or externally as side-by-side extensions running on SAP BTP.
- Extensions can only use released local or remote public SAP Application programming Interfaces (APIs) and BADIs as well as ABAP RESTful application programming model Business Object extension points.
- Extension can only be implemented using cloud enabled and released technology.

* Extension Types :-

SAP S/4 HANA cloud extensibility model are the extension types.

i) On stack extensions

- Key User Extensions
- Developer extensions

ii) Side-By-Side extensions.

Key User Extensibility.

The rise of "citizen developers" has changed how applications are built. These are people who use easy-to-use tools (low-code/no-code) to create apps without needing deep technical knowledge. This group includes business experts and key users.

- Key user extensibility means allowing these users to make simple changes or add features to SAP applications using web-based tools called SAP Fiori apps.

These tools are designed to be quick and easy to use, and they work with all version of SAP S/4 HANA.

The goal is to let users make changes without affecting the core system.

* Samm Tools and capabilities

1. UI Adaptation at Runtime -

Key users can change the user interface at runtime.

For eg- renaming field, adding new fields.

2. Custom fields- key users can create and edit custom fields for SAP tables and services.

3. Custom Business Logic - key users can add new business logic to SAP S/4 HANA cloud using a web-based editor, without needing to know the technical details.

4. Custom Business object (BO) -

Key users can create entirely new business objects and their database tables, along with APIs for data operations.

5. custom cds view -

Key users can model data access in ways that existing views may not support, aiding in reporting and custom logic.

6. Custom Analytical Queries -

Key users can easily create queries for reporting based on SAP or custom data sources.

7. KPIs - key user can design new key performance indicators (KPIs) and visualize them as tiles in the SAP Fiori launchpad.

8. FORMS - Key users can create form and email templates based on SAP data sources

These tools empower key users to make significant customizations without needing deep tech knowledge.

Developer Extensions -

Developer extensions are designed for traditional ABAP developers.

These extensions allow developers to handle more complex tasks, such as.

- Accessing data frequently and/or using complex SQL queries.
- Running extensions within the same transaction as SAP applications.
- Creating Remote APIs for other applications.

These extensions are built within the SAP S/4 HANA cloud using the ABAP cloud development model, which includes tools like ABAP development tools for eclipse & public interfaces.

There are two main types of developer extensions:

1. ABAP RESTful application programming model extensibility option

2. Business Add-Ins (BADIs)

125-business object interface

800 Business Add-In

Business Add-ins (BADIs)

- BADIs are a way to enhance SAP applications and have been around for a long time.
- They are designed to be separate from the main SAP system, so updates to SAP won't affect them.
- BADI's are object-oriented and follow the clean core concept.

System Landscape

- Originally SAP S/4HANA cloud had two environments: one for development / testing and one for production.
- For more complex developer extensions, a third environment for development is added, similar to traditional setups.
- Both key user and developer extensions are tested before being used in production.

ABAP RESTful Application Programming Model Extensibility

- This model allows developers to extend various parts of the system in a stable way.
- Extensions can be made at different layers, such as data models, field behaviors, and service definitions.
- For data models, developers can add new fields to database tables and CDS views, which are explicitly marked as extensible by SAP.

Field Behavior and Behavior Extensions:

- Developers can add new fields to existing business objects (BOs) and use them in mappings.
- They can create new reactions, rules and validations for these BOs.
- Additional save actions can be defined.
- For draft-enabled BOs, new actions can be added, and existing ones can be extended.

Node and service definition Extensions:

- Developers can add new entities (like tables) to existing BOs.
- These new entities can use the same field and behavior extensions.
- To make these new entities available for use, service definitions can be extended.

Extensibility Enablement VS Extensibility Provisioning

Extensibility Enablement:

SAP decides if and how extensions can be made.

Extensibility Provisioning -

customers actually create and use the extensions.

Side-by-side Extensions

- These extensions are for traditional developers and run on SAP Business Technology platform (BTP) not on SAP S/4 HANA cloud.
- They can use various technologies like Java, Node.js or ABAP
- These extensions access SAP S/4 HANA cloud data through remote API but don't need to run within the SAP S/4 HANA cloud environment.

Use cases for side-by-side Extensions:

- custom applications that need SAP BTP services like machine learning or AI
- Applications that integrate with multiple systems, both on-premise and cloud based
- SaaS solutions developed by partners.

SAP BTP, ABAP environment :-

- This is ^{SAP BTP} SAP's Platform as a service (PaaS) for ABAP development.
- It support both side-by-side and standalone applications
- It includes an SAP HANA database and uses the ABAP RESTful application programming model. (RAP)
- Developers use ABAP development tools for Eclipse and the ABAP cloud language option.

This setup allows developers to create advanced applications that can interact with SAP S/4 HANA cloud data while running on SAP BTP.

Future Proof Extensions:

- SAP S/4 HANA cloud ensure extensions are future-proof by using a specific list of stable artifacts and object called public released interfaces.
- These interfaces include local and remote APIs and extension points, which SAP guarantees will remain stable.

Local vs Remote APIs

Local APIs - Available within the SAP S/4 HANA cloud layer for key user or developer extensions.

Remote APIs - Used by SAP BTP, ABAP environment for side-by-side extensions, as it doesn't have access to Local APIs

This setup ensures that extensions are stable and can adapt to future updates without issues.

To choose right extensibility option, consider these points:-

1. Extension use case -

- Is it a new application or an extension to an existing SAP app?

2. Extension Architecture -

- Is the extension loosely or tightly connected to SAP S/4 HANA cloud?

- Does it use only SAP S/4 HANA cloud data, or does it integrate data from multiple sources?

3. Extension scope and size:

- who is providing the extension and for what purpose?
- Is it a small extension by key users, a custom development project by a development team, or a partner SaaS application for many customers?

SAP S/4 HANA Cloud Extensibility Rules:-

- You can't modify SAP objects.
- Extensions must be separate from SAP code.
- Only use approved APIs and extension points.

Benefits -

Following these rules ensures smooth software upgrades.

SAP S/4 HANA Cloud Public Edition:

Updates automatically for all customers at the same time

SAP S/4 HANA Cloud Private Edition and On-premise

- Updates are done through traditional, customer-specific projects
- These versions have more features than the Public Edition, allowing for additional use cases.

Recommendations

- Use the ABAP cloud development model as much as possible, especially if planning to migrate to the public edition.
- This model ensures smooth migrations and minimal disruption

Classic Extensibility:

- Still available if the ABAP cloud model can't be used.
- However, it may cause problems & make future upgrades more difficult and expensive

Tier 1: Cloud Extensibility Model

- Based on ABAP cloud development model
- Key Points
 - Only approved ABAP cloud object types can be developed.
 - Enforced use of ABAP cloud language and released APIs (publicly available API)
 - Development done using ABAP tools for Eclipse.

Usage:

- Available for both Public and Private-Edition customers.
- Encouraged as the first option for development
- Supports key user extensions, on-stack developer extensions and side-by-side extensions on SAP BTP.

Tier 2: cloud API Enablement

- For Private Edition customers only.

Key points -

- Used when non-released API needed
- Involves creating "custom wrappers" around non-released APIs. wraps released api around non
- Follows Tier 1 rules but uses ABAP test cockpit checks for compliance.

Usage:

- Allows use of non-released APIs by wrapping them in customer-created APIs.
- Encouraged to request missing APIs through the customer Influence channel
- Over time, more APIs will be moved to the public released category.

Tier 3 : classic ABAP extensions

For custom ABAP code not possible with Tier 1 or Tier 2.

Key Points:

- No restrictions on ABAP language or object types.
- Carries the highest risk of software upgrade disruptions

Usage:

- should be avoided if possible
- Use customer includes for DDIC extensions, CDS extends for CDS extensions; and redefine ODATA services.
- Non-released BADIs, user exits, and customer exists can be used but should be analyzed after updates.
- Avoid implicit enhancement spots and modification unless absolutely necessary.

* Examples of Use Cases -

Tier 1:

- Adding custom fields to a database table or CDS view.
- Implementing a released SAP BADI
- Creating a custom ABAP RESTful application programming model-based SAP Fiori app.

Tier 2:-

- creating a wrapper class around non-released SAP objects.
- creating a wrapper CDS view for non-released SAP tables or views.
- creating an ABAP RESTful application programming model interface around non-released SAP objects.

Tier 3:-

- Implementing a non-released BADI
- Extending an SAP Fiori app based on legacy technologies.
- Modifying any SAP object